

Università degli Studi di Camerino

SCUOLA DI SCIENZE E TECNOLOGIE

Corso di Laurea in Informatica (Classe L-31)

Ingegneria del software
Filiera agricola

Studenti

Poian Filippo

Vincenzo Mattioli

Tiziano Chiarini

Docenti

Polini Andrea

Morichetta Andrea

Matricole 123164, 123014, 123024

A.A. 2025/2026

Sommario

Il presente documento illustra l'architettura, le scelte progettuali e i dettagli implementativi della piattaforma *FilieraAgricola*. Il sistema è una soluzione Enterprise progettata per la gestione digitale della filiera agroalimentare, con l'obiettivo di garantire la tracciabilità completa dei prodotti ("dal campo alla tavola"), certificare la qualità tramite processi di revisione e abilitare un canale di vendita diretto (Marketplace B2C).

La relazione analizza lo stack tecnologico (Java 21, Spring Boot 3), i Design Patterns applicati per risolvere complessità di business (State, Strategy, Factory) e la modellazione dei dati relazionali, offrendo una panoramica completa del ciclo di sviluppo del software, dall'analisi dei requisiti al testing.

Indice

Sommario	2
1 Introduzione e Contesto	1
1.1 Scenario di Riferimento	1
1.2 Obiettivi del Progetto	1
1.3 Ambito del Sistema (Scope)	1
2 Analisi dei Requisiti	2
2.1 Attori e Ruoli	2
2.2 Requisiti Funzionali Core	2
3 Architettura del Sistema	3
3.1 Stack Tecnologico	3
3.2 Stile Architetture	3
3.3 Flusso di Richiesta (Request Lifecycle)	4
4 Ingegneria del Software e Design Patterns	5
4.1 Gestione del Workflow (State Pattern)	5
4.2 Logiche di Pricing Dinamico (Strategy & Factory)	5
4.3 Data Transfer Object (DTO)	6
5 Modellazione dei Dati e Persistenza	7
5.1 Schema Entità-Relazione	7
5.2 Il Grafo di Tracciabilità (Traceability Graph)	7
5.2.1 Implementazione della Relazione Autoreferenziale	7
5.3 Gestione delle Transazioni e Integrità dei Dati	8
5.3.1 Il Paradigma ACID	8
5.3.2 Propagazione delle Modifiche (Side-Effect Management)	9
5.4 Gestione delle Dipendenze e Consistenza dei Pacchetti	10
5.4.1 Implementazione dell'Invariante di Creazione	10
5.4.2 Propagazione delle Modifiche (Side-Effect Management)	11
6 Sicurezza e Controllo Accessi	12
6.1 Autenticazione Stateless (JWT)	12
6.1.1 Il Flusso di Autenticazione	12
6.2 Autorizzazione Granulare	12
6.2.1 Livello Endpoint (RBAC)	12
6.2.2 Livello Oggetto (ACL e Ownership)	13
7 Visual Paradigm	14
7.1 Analisi dei Casi d'Uso (Use Case Diagram)	14

7.1.1	Identificazione degli Attori del Sistema	14
7.1.2	Casi d'Uso Principali	14
7.1.3	Scenari di Business	14
7.2	Analisi dei Casi d'Uso (Use Case Diagram)	15
7.2.1	Identificazione degli Attori del Sistema	15
7.2.2	Casi d'Uso Principali	15
7.2.3	Scenari di Business	16
7.3	Diagramma delle Classi Implementate	16
7.3.1	Architettura dei Controller	16
7.3.2	Modellazione delle Entità e Relazioni	16
7.4	Diagrammi di Interazione (Sequence Diagrams)	17
7.4.1	Processo di Autenticazione JWT: Flusso di comunicazione per l'ac- cesso sicuro	17
7.4.2	Gestione Dinamica dell'Ordine: Logica di aggiornamento dello stato	18
7.5	Design Patterns nel Modello Visuale	18
7.5.1	Strategie di Prezzo (Strategy Pattern)	18
7.5.2	Ciclo di Vita e Stati: Modellazione delle Entità	19
8	Testing, Conclusioni e Sviluppi Futuri	20
8.1	Strategia di Testing e Validazione	20
8.1.1	Configurazione dell'Ambiente	20
8.2	Conclusioni	20
8.3	Sviluppi Futuri	20

1. Introduzione e Contesto

1.1 Scenario di Riferimento

Il settore agroalimentare moderno è caratterizzato da una forte frammentazione informativa. I consumatori finali hanno scarsa visibilità sull'origine reale dei prodotti, mentre i produttori locali faticano a valorizzare la qualità delle loro materie prime una volta che queste entrano nei processi di trasformazione industriale. La mancanza di trasparenza riduce la fiducia del mercato e non premia gli attori virtuosi della filiera.

1.2 Obiettivi del Progetto

Il progetto *FilieraAgricola* nasce per colmare questo divario informativo tramite una piattaforma centralizzata che persegue tre obiettivi principali:

1. **Tracciabilità Immutabile:** Mappare digitalmente ogni passaggio di stato della merce, garantendo la genealogia del prodotto attraverso la registrazione delle trasformazioni (input/output).
2. **Qualità Certificata (Curation):** Introdurre un livello di revisione umana sui contenuti pubblicati, per evitare frodi o informazioni inesatte.
3. **Commercializzazione Diretta:** Offrire un Marketplace integrato che permetta la vendita di prodotti singoli e bundle promozionali (pacchetti), disintermediando il rapporto produttore-consumatore.

1.3 Ambito del Sistema (Scope)

Il progetto si focalizza sullo sviluppo dell'infrastruttura **Backend RESTful**, che costituisce il nucleo logico-funzionale della piattaforma. Il perimetro di implementazione comprende:

- La gestione sicura degli accessi tramite protocollo RBAC (Role-Based Access Control).
- Il motore di tracciabilità profonda per la filiera agroalimentare.
- I moduli di e-commerce (Catalogo, Carrello, Ordini) e la gestione territoriale (Eventi e Mappe).

2. Analisi dei Requisiti

2.1 Attori e Ruoli

Il sistema implementa un modello di controllo degli accessi basato sui ruoli (RBAC), definiti nell'enumerativo `ERole`. Un singolo utente può possedere molteplici ruoli simultaneamente, permettendo una flessibilità operativa elevata.

- **Gestore della Piattaforma:** Amministratore tecnico, responsabile dell'accREDITamento delle aziende e della manutenzione del sistema.
- **Curatore:** Figura garante della qualità. Ha il compito di approvare o rifiutare Prodotti ed Eventi prima che diventino visibili al pubblico.
- **Produttore:** Azienda agricola che inserisce materie prime (Lotti base) nel sistema.
- **Trasformatore:** Azienda che processa materie prime per creare nuovi prodotti (Lotti derivati).
- **Distributore:** Gestisce la logistica finale verso il consumatore.
- **Buyer (Utente Finale):** Naviga il catalogo, visualizza la filiera e acquista prodotti o prenota eventi.

2.2 Requisiti Funzionali Core

1. **Workflow Approvativo:** Nessun contenuto (Prodotto/Evento) può essere pubblicato senza revisione esplicita.
2. **Tracciabilità Multi-Livello (Deep Traceability):** Il sistema deve garantire la ricostruzione completa della genealogia di un prodotto. Dato un lotto di prodotto finito, il sistema deve essere in grado di attraversare ricorsivamente le relazioni di trasformazione per identificare tutti i lotti di materie prime e semilavorati che hanno contribuito alla sua composizione, risalendo l'intera catena produttiva fino all'origine ("dal campo alla tavola").
3. **Acquisto Multi-tipologia:** Il carrello deve supportare sia prodotti singoli che pacchetti composti, calcolando i prezzi in modo differenziato.
4. **Geolocalizzazione:** Visualizzazione su mappa degli attori della filiera e degli eventi.

3. Architettura del Sistema

La progettazione del sistema *FilieraAgricola* è stata guidata dai principi di modularità, scalabilità e manutenibilità. L'architettura software adottata mira a disaccoppiare le diverse responsabilità applicative, garantendo che le logiche di presentazione, di business e di persistenza evolvano in modo indipendente.

3.1 Stack Tecnologico

La selezione dello stack tecnologico ha privilegiato standard Enterprise moderni e versioni con supporto a lungo termine (LTS), per garantire la stabilità del sistema nel tempo.

- **Linguaggio Core: Java 21 (LTS).** Questa scelta permette di sfruttare le moderne caratteristiche del linguaggio, come i `Record` (utilizzati estensivamente per i DTO immutabili) e il `Pattern Matching`, mantenendo al contempo una compatibilità industriale garantita per diversi anni.
- **Framework Applicativo: Spring Boot 3.4.5.** Framework de-facto per lo sviluppo Java Enterprise, scelto per il suo approccio *Convention over Configuration* e per l'ecosistema maturo.
 - *Spring Web*: Fornisce lo stack per creare API RESTful e gestisce il container Servlet embedded (Tomcat).
 - *Spring Data JPA*: Astrae la complessità delle query SQL, fornendo un'interfaccia ad alto livello basata su Hibernate per l'accesso ai dati.
 - *Spring Security*: Gestisce l'autenticazione e l'autorizzazione dichiarativa.
- **Database e Persistenza:** È stata adottata una strategia a doppio database per separare gli ambienti:
 - **MySQL (Produzione)**: RDBMS solido e performante per garantire la persistenza durevole dei dati e il supporto alle transazioni complesse.
 - **H2 Database (Testing)**: Database in-memory volatile. Viene utilizzato durante l'esecuzione dei test automatici per garantire velocità e isolamento: ad ogni avvio dei test, lo schema viene ricreato da zero (`create-drop`), assicurando un ambiente pulito e predicibile.
- **Build and Dependency Management: Maven.** Utilizzato per la gestione delle dipendenze e il ciclo di vita del build. L'uso del *Maven Wrapper* garantisce la riproducibilità della compilazione su qualsiasi ambiente, indipendentemente dalla versione di Maven installata localmente.

3.2 Stile Architeturale

Il sistema adotta una **Layered Architecture** (Architettura a Strati) classica. Questo pattern impone una rigorosa separazione delle responsabilità (*Separation of Concerns*), dove ogni livello comunica solo con quello immediatamente sottostante.

1. **Presentation Layer (controller)**: Rappresenta l'interfaccia verso il mondo esterno (API REST). I controller sono responsabili di:
 - Ricevere le richieste HTTP e validare i payload in ingresso (DTO) tramite annotazioni come `@Valid`.
 - Invocare i servizi di business appropriati.
 - Tradurre le risposte (o le eccezioni) in codici di stato HTTP standard (200 OK, 400 Bad Request, ecc.).
2. **Service Layer (service)**: Costituisce il cuore applicativo. Questo strato è agnostico rispetto al protocollo web e incapsula la logica di dominio. È il livello dove vengono definiti i **Confini Transazionali**: ogni metodo pubblico di un Service rappresenta un'unità di lavoro atomica (ACID), gestita tramite l'annotazione `@Transactional`.
3. **Persistence Layer (repository)**: Strato di accesso ai dati. Le interfacce estendono `JpaRepository`, permettendo di eseguire operazioni CRUD e query complesse senza scrivere SQL manuale, delegando l'interazione con il database all'ORM (Hibernate).

3.3 Flusso di Richiesta (Request Lifecycle)

L'architettura del sistema impone uno schema di gestione delle richieste uniforme per tutti gli endpoint. Di seguito viene illustrato, come esempio paradigmatico, il ciclo di vita di una richiesta di “**Creazione Ordine**”, scelto per la sua completezza in quanto attraversa tutti i layer architetturali.

1. **Richiesta Client**: Il client invia una `POST /api/ordini` contenente il JSON dell'ordine e l'Header `Authorization: Bearer <token>`.
2. **Security Filter Chain**: Prima di raggiungere il codice applicativo, la richiesta viene intercettata dal `JwtAuthFilter`. Questo componente valida la firma crittografica del token JWT, estrae l'identità dell'utente e popola il `SecurityContext` di Spring.
3. **Controller (Mapping & Validazione)**: Il `OrdineController` riceve la richiesta. Il framework Jackson deserializza il JSON nel `OrdineRequest` (DTO). Viene eseguita la validazione formale (es. controllo che la lista prodotti non sia vuota).
4. **Service (Logica Core)**: Il controller delega l'elaborazione a `OrdineService`. Qui avvengono le operazioni critiche:
 - Verifica della disponibilità dei prodotti.
 - Calcolo dei prezzi tramite le `PricingStrategy`.
 - Applicazione delle regole di business (es. sconti pacchetto).
5. **Persistence (Salvataggio Atomico)**: Il Service invoca i Repository per salvare le entità `Ordine` e `OrderLine`. Hibernate traduce queste operazioni in istruzioni SQL `INSERT` all'interno di una transazione database aperta dal Service.
6. **Risposta o Gestione Errore**: Se tutto va a buon fine, il sistema restituisce un DTO di risposta con HTTP 200. In caso di errore (es. prodotto non trovato), l'eccezione risale fino al `GlobalExceptionHandler`, che la trasforma in un messaggio JSON di errore (es. HTTP 404 o 400) comprensibile al client.

4. Ingegneria del Software e Design Patterns

Uno degli aspetti più rilevanti del progetto è l'uso di Design Patterns per gestire la complessità e garantire manutenibilità.

4.1 Gestione del Workflow (State Pattern)

Il ciclo di vita dei contenuti (ContentSubmission) segue stati rigidi: *Bozza*, *In Revisione*, *Approvato*, *Rifiutato*.

- **Problema:** Evitare catene di if-else nei Service per gestire le transizioni (es. impedire modifica se *In Revisione*).
- **Soluzione:** Implementazione dello **State Pattern**. L'interfaccia ContentState definisce le azioni (sottometti, approva, rifiuta).

```
1 public interface ContentState {  
2     void sottometti(ContentSubmission context);  
3     void approva(ContentSubmission context);  
4     void rifiuta(ContentSubmission context, String motivazione);  
5 }
```

Codice 4.1: Interfaccia ContentState

4.2 Logiche di Pricing Dinamico (Strategy & Factory)

Una delle complessità gestite dal sistema riguarda il calcolo del prezzo finale delle righe d'ordine. Poiché un ordine può contenere entità eterogenee (singoli prodotti o pacchetti promozionali), la logica di calcolo non può essere uniforme.

Per risolvere questa problematica senza violare il principio di responsabilità singola (SRP), sono stati combinati due pattern comportamentali e creazionali:

- **Strategy Pattern:** Consente di incapsulare algoritmi di calcolo intercambiabili all'interno di una famiglia di classi. L'interfaccia PricingStrategy definisce il contratto calculatePrice, implementato concretamente da:
 - SingleItemPricingStrategy: Restituisce il prezzo base del singolo articolo.
 - PackagePricingStrategy: Implementa una logica di aggregazione, sommando i prezzi dei componenti del pacchetto ed eventualmente applicando scontistiche dedicate.
- **Factory Pattern:** Utilizzato per disaccoppiare la logica di selezione dell'algoritmo dal codice client (OrdineService). La classe PricingStrategyFactory analizza a runtime il tipo dell'oggetto contenuto nella OrderLine e restituisce l'istanza della strategia appropriata.

Questa architettura garantisce il rispetto dell'**Open/Closed Principle**: in futuro sarà possibile introdurre nuove politiche di prezzo (es. *ScontoStagionaleStrategy*) aggiungendo nuove classi senza modificare il codice esistente del servizio ordini.

```
1 @Component
2 public class PricingStrategyFactory {
3
4     private final SingleItemPricingStrategy singleItemStrategy;
5     private final PackagePricingStrategy packageStrategy;
6
7     // Injection dei Bean delle strategie tramite costruttore
8     public PricingStrategyFactory(SingleItemPricingStrategy singleItemStrategy,
9                                   PackagePricingStrategy packageStrategy) {
10         this.singleItemStrategy = singleItemStrategy;
11         this.packageStrategy = packageStrategy;
12     }
13
14     /**
15      * Seleziona la strategia di prezzo in base al tipo dinamico dell'item.
16      */
17     public PricingStrategy getStrategy(OrderLine line) {
18         // Controllo polimorfico sul contenuto della riga d'ordine
19         if (line.getItem() instanceof Pacchetto) {
20             return packageStrategy;
21         }
22         // Default per prodotti semplici
23         return singleItemStrategy;
24     }
25 }
```

Codice 4.2: Implementazione della PricingStrategyFactory

4.3 Data Transfer Object (DTO)

Non vengono mai esposte le Entity JPA direttamente nelle API. Si utilizzano oggetti DTO per nascondere dati sensibili, prevenire ricorsioni infinite nella serializzazione JSON e disaccoppiare il contratto API dalla struttura del Database.

5. Modellazione dei Dati e Persistenza

5.1 Schema Entità-Relazione

Il modello dati sfrutta la potenza di JPA per rappresentare relazioni complesse e garantire l'integrità referenziale. Le scelte modellistiche principali includono:

- **Utente:** Relazione @ManyToMany con Role (per la gestione multi-ruolo) e @OneToOne con UserProfile (per separare le credenziali dai dati anagrafici).
- **Ereditarietà Logica:** Tabelle separate per i profili aziendali (ActorProfile_Produttore, ActorProfile_Trasformatore, ecc.) collegate all'utente. Questa scelta permette di salvare dati specifici per ogni tipo di attore (es. Partita IVA vs Codice Fiscale) mantenendo un'unica identità di login.

5.2 Il Grafo di Tracciabilità (Traceability Graph)

Il cuore del sistema di tracciabilità è rappresentato dall'entità ProductBatch (Lotto). A differenza di sistemi semplificati che gestiscono la filiera come una lista sequenziale, *FilieraAgricola* adotta un modello dati basato su un **Grafo Diretto Aciclico (DAG)**.

Questa scelta progettuale è necessaria per rappresentare fedelmente i processi di trasformazione industriale, dove:

- Un lotto di output può derivare dalla combinazione di più lotti di input (es. una *Passata di Pomodoro* composta da lotti di pomodori e sale).
- Un singolo lotto di materia prima può essere suddiviso per produrre diversi lotti di output (Branching).
- Il grafo è **aciclico** in quanto la trasformazione segue la freccia del tempo: un prodotto finito non può essere ingrediente di se stesso o dei suoi predecessori.

5.2.1 Implementazione della Relazione Autoreferenziale

A livello di persistenza JPA, la relazione multi-a-molti tra i lotti non è stata mappata con una semplice @ManyToMany, bensì è stata esplosa utilizzando un'entità di giunzione dedicata: BatchInputLink.

Questa tecnica, nota come **Associazione con Attributi**, è stata fondamentale per storicizzare non solo *quale* lotto è stato usato, ma anche *in che quantità*.

- **Nodi del Grafo:** Rappresentati dall'entità ProductBatch.
- **Archi del Grafo:** Rappresentati dall'entità BatchInputLink, che collega un outputBatch (figlio) a un inputBatch (genitore), qualificando la relazione con l'attributo quantity.

Di seguito viene mostrato lo schema logico delle classi Java che realizzano questa struttura:

```

1 @Entity
2 public class ProductBatch {
3     @Id
4     private Long id;
5
6     // Lotti che hanno contribuito a creare questo batch (Genitori)
7     @OneToMany(mappedBy = "outputBatch")
8     private List<BatchInputLink> inputBatches;
9
10    // Lotti creati a partire da questo batch (Figli)
11    @OneToMany(mappedBy = "inputBatch")
12    private List<BatchInputLink> outputBatches;
13 }
14
15 @Entity
16 public class BatchInputLink {
17     // Il lotto "Genitore" (es. Cassetta di Pomodori)
18     @ManyToOne
19     @JoinColumn(name = "input_batch_id")
20     private ProductBatch inputBatch;
21
22     // Il lotto "Figlio" (es. Passata di Pomodoro)
23     @ManyToOne
24     @JoinColumn(name = "output_batch_id")
25     private ProductBatch outputBatch;
26
27     // Attributo della relazione: quanta materia prima e' stata usata?
28     private Double quantity;
29 }

```

Codice 5.1: Modellazione della relazione autoreferenziale tramite BatchInputLink

Questa struttura dati permette al TracciabilitaService di eseguire algoritmi di visita del grafo (Depth-First Search) per ricostruire l'intera genealogia del prodotto, navigando dai nodi foglia (prodotti finiti) fino alle radici (materie prime).

5.3 Gestione delle Transazioni e Integrità dei Dati

In un sistema Enterprise come *FilieraAgricola*, la consistenza dei dati è un requisito non funzionale critico. Per garantire che le operazioni di modifica del database avvengano in modo sicuro e predicibile, è stato adottato un approccio dichiarativo alla gestione delle transazioni tramite il framework Spring.

5.3.1 Il Paradigma ACID

Il sistema aderisce alle proprietà **ACID** (Atomicity, Consistency, Isolation, Durability). In particolare, l'attenzione progettuale si è concentrata sull'**Atomicità**: un insieme di operazioni deve essere eseguito nella sua interezza o non essere eseguito affatto (approccio *All-or-Nothing*).

L'annotazione `@Transactional` è applicata ai metodi del Service Layer che coinvolgono scritture su tabelle multiple. Spring gestisce i **confini transazionali** (Transaction Boundaries) in modo dichiarativo: il framework apre una transazione all'ingres-

so del metodo e ne esegue il commit all'uscita, oppure intercetta le eccezioni runtime (`RuntimeException`) per attivare il rollback automatico in caso di errore.

5.3.2 Propagazione delle Modifiche (Side-Effect Management)

Il sistema deve gestire anche il caso dinamico in cui un prodotto, precedentemente valido, venga modificato dopo essere stato inserito in un pacchetto già pubblicato.

Per rispettare l'invariante di modifica, è stato implementato un meccanismo di **Propagazione dello Stato**. Quando un `Produttore` modifica un prodotto:

1. Il sistema identifica tutti i pacchetti che contengono quel prodotto tramite una query inversa sulla tabella di giunzione (`MarketplaceItemPacchetto`).
2. I pacchetti identificati subiscono una **regressione di stato forzata**: da `APPROVATO` vengono riportati a `BOZZA` (o `IN_REVISIONE`).
3. I pacchetti vengono rimossi temporaneamente dal Marketplace fino a nuova approvazione da parte del Curatore.

Questo approccio previene incongruenze commerciali, garantendo che non vengano venduti pacchetti contenenti prodotti non più disponibili o con caratteristiche (es. prezzo) non allineate.

5.4 Gestione delle Dipendenze e Consistenza dei Pacchetti

Il modulo *Marketplace* introduce l'entità *Pacchetto* (o *Bundle*), definita come un aggregato commerciale di più prodotti venduti unitamente a un prezzo promozionale. Dal punto di vista architetturale, questa entità introduce una forte dipendenza logica tra il contenitore (il *Pacchetto*) e i contenuti (i *Prodotti*), che richiede una gestione rigorosa della consistenza dei dati.

Il *PacchettoService* implementa una logica di validazione basata sul mantenimento di due ****Invarianti di Stato**** fondamentali:

1. **Invariante di Qualità (Creation Invariant):** Un pacchetto non può essere creato né pubblicato se anche uno solo dei suoi componenti non possiede lo stato *APPROVATO*. Questa regola impedisce che merci non certificate o in fase di bozza vengano commercializzate tramite canali indiretti (bundle).
2. **Invariante di Sincronizzazione (Modification Invariant):** La validità di un pacchetto è legata alla validità dei suoi componenti. Se un prodotto componente subisce una modifica critica (es. cambio prezzo, ritiro dal mercato), tale evento deve propagarsi al pacchetto, invalidandone lo stato di approvazione.

5.4.1 Implementazione dell'Invariante di Creazione

Durante la fase di creazione o aggiornamento di un pacchetto, il sistema esegue un controllo puntuale sullo stato di ogni articolo candidato all'inserimento. L'operazione è atomica: il fallimento della validazione su un singolo item impedisce il salvataggio dell'intero pacchetto.

```

1 @Transactional
2 public Pacchetto creaPacchetto(PacchettoRequest request, Utente curatore) {
3     Pacchetto pacchetto = new Pacchetto();
4     // ... mapping dei dati anagrafici ...
5
6     for (Long itemId : request.getItemsIds()) {
7         MarketplaceItem item = itemRepository.findById(itemId)
8             .orElseThrow(() -> new EntityNotFoundException("Item non trovato"));
9
10        // Verifica che il prodotto sottostante sia certificato (APPROVATO).
11        if (!isItemApproved(item)) {
12            throw new InvalidBundleException(
13                "Violazione di consistenza: il prodotto " + item.getNome() +
14                " non e' approvato e non puo' essere aggiunto al pacchetto."
15            );
16        }
17
18        pacchetto.addItem(item);
19    }
20
21    // Il pacchetto nasce sempre in stato BOZZA per richiedere una nuova
22    // approvazione specifica sulla combinazione dei prodotti.
23    return pacchettoRepository.save(pacchetto);
24 }
25
26 private boolean isItemApproved(MarketplaceItem item) {
27     return item.getContenuto().getStato().equals(StatoContenuto.APPROVATO);
28 }

```

Codice 5.2: Logica di validazione e consistenza nel *PacchettoService*

Questa logica garantisce che la proprietà di "qualità certificata" sia transitiva: se i componenti sono validi, allora il pacchetto è tecnicamente valido per essere sottoposto a revisione.

5.4.2 Propagazione delle Modifiche (Side-Effect Management)

Il sistema deve gestire anche il caso dinamico in cui un prodotto, precedentemente valido, venga modificato dopo essere stato inserito in un pacchetto già pubblicato.

Per rispettare l'invariante di modifica, è stato implementato un meccanismo di ****Propagazione dello Stato****. Quando un `Produttore` modifica un prodotto:

1. Il sistema identifica tutti i pacchetti che contengono quel prodotto tramite una query inversa sulla tabella di giunzione.
2. I pacchetti identificati subiscono una regressione di stato forzata: da `APPROVATO` vengono riportati a `BOZZA` (o `IN_REVISIONE`).
3. I pacchetti vengono rimossi temporaneamente dal Marketplace fino a nuova approvazione da parte del Curatore.

Questo approccio previene incongruenze commerciali, come la vendita di un pacchetto contenente prodotti non più disponibili o con prezzi non aggiornati.

6. Sicurezza e Controllo Accessi

La sicurezza del sistema è stata progettata seguendo il principio della *Defense in Depth* (difesa in profondità), implementando controlli su molteplici livelli, dalla validazione del token di accesso fino alla verifica puntuale della proprietà dei dati.

6.1 Autenticazione Stateless (JWT)

A differenza delle architetture web tradizionali basate su sessioni server-side (dove il server deve memorizzare lo stato di login di ogni utente in memoria), *FilieraAgricola* adotta un approccio **Stateless** basato su standard **JWT (JSON Web Token)**.

Questa scelta architetturale offre vantaggi significativi in termini di scalabilità: il server non deve mantenere alcuno stato, permettendo al backend di essere replicato orizzontalmente senza problemi di sincronizzazione delle sessioni.

6.1.1 Il Flusso di Autenticazione

Il meccanismo è orchestrato dal filtro personalizzato `JwtAuthFilter`, che si inserisce nella catena di sicurezza di Spring (*Security Filter Chain*). Il ciclo di vita di una richiesta autenticata è il seguente:

1. **Estrazione:** Il filtro intercetta ogni richiesta HTTP in arrivo e ispeziona l'header `Authorization` alla ricerca di una stringa nel formato `Bearer <token>`.
2. **Validazione:** Il token viene decodificato utilizzando la chiave segreta del server (tramite la classe di utilità `JwtUtils`) per verificarne la firma digitale e la scadenza temporale.
3. **Autenticazione nel Contesto:** Se il token è valido, il filtro estrae lo username, carica i dettagli dell'utente e popola il `SecurityContextHolder`. Da questo momento in poi, Spring considera l'utente autenticato per la durata della singola richiesta.

6.2 Autorizzazione Granulare

L'autenticazione verifica *chi* è l'utente, ma l'autorizzazione decide *cosa* può fare. Il sistema implementa due livelli distinti di controllo.

6.2.1 Livello Endpoint (RBAC)

Il primo livello di difesa è il **Role-Based Access Control (RBAC)**. Tramite le annotazioni dichiarative di Spring Security, viene limitato l'accesso ai metodi dei Controller in base al ruolo posseduto dall'utente.

Esempio:

```
@PreAuthorize("hasRole('CURATORE')")
```


Questa annotazione garantisce che solo gli utenti con il ruolo di CURATORE possano invocare l'endpoint sottostante. Se un PRODUTTORE tentasse l'accesso, il framework restituirebbe automaticamente un errore 403 Forbidden.

6.2.2 Livello Oggetto (ACL e Ownership)

Il controllo basato sui ruoli non è sufficiente per prevenire vulnerabilità di tipo *Insecure Direct Object Reference (IDOR)*. Un utente potrebbe avere il ruolo corretto (es. PRODUTTORE) per chiamare l'API di modifica lotto, ma tentare di modificare un lotto creato da un altro produttore semplicemente indovinandone l'ID.

Per mitigare questo rischio, è stato implementato un servizio di sicurezza custom (CustomSecurityService) che verifica la **Ownership** (proprietà) della risorsa.

```

1 @Service("customSecurity")
2 public class CustomSecurityService {
3
4     // Verifica che l'ID passato nella richiesta appartenga all'utente loggato
5     public boolean isResourceOwner(Authentication auth, Long resourceId) {
6         String currentUsername = auth.getName();
7         // Logica di recupero risorsa e confronto proprietario...
8         return resourceOwner.equals(currentUsername);
9     }
10 }

```

Codice 6.1: Controllo di Ownership nel Service Layer

Questa logica viene integrata direttamente nelle annotazioni di sicurezza, permettendo espressioni complesse come:

```

@PreAuthorize("hasRole('PRODUTTORE') and
@customSecurity.isResourceOwner(authentication, #id)")

```

In questo modo, il metodo viene eseguito solo se l'utente ha il ruolo corretto **E** se è l'effettivo proprietario del dato che sta cercando di manipolare.

7. Visual Paradigm

7.1 Analisi dei Casi d'Uso (Use Case Diagram)

Il diagramma dei casi d'uso modella le interazioni tra gli attori esterni e il sistema, definendo i confini funzionali della piattaforma "Filiera Agricola". La modellazione visuale ha permesso di mappare i requisiti utente in azioni atomiche supportate dallo stack Spring Boot.

7.1.1 Identificazione degli Attori del Sistema

Gli attori identificati nel progetto Visual Paradigm rappresentano i ruoli e i sistemi esterni che interagiscono con il perimetro applicativo:

- **Acquirente:** Utente finale che consulta il catalogo, gestisce il proprio carrello, effettua ordini e prenota la partecipazione agli eventi.
- **Animatore della Filiera:** Attore responsabile della promozione del territorio; propone la creazione di nuovi eventi soggetti a validazione.
- **Gestore della Piattaforma:** Amministratore di sistema con privilegi per l'aggiornamento dello stato logistico degli ordini (es. `IN_ELABORAZIONE`, `SPEDITO`).
- **Utente Generico:** Rappresenta l'attore non autenticato i cui casi d'uso sono limitati alla registrazione e alla consultazione delle informazioni pubbliche.
- **Sistemi Esterni:** Il modello include il **Sistema OSM** (OpenStreetMap) per la proiezione geospaziale delle aziende e i **Sistemi Social** per la condivisione dei contenuti di filiera.

7.1.2 Casi d'Uso Principali

Il diagramma visuale raggruppa le funzionalità core in base agli obiettivi degli attori:

- **Autenticazione (UC12):** Gestisce il flusso di accesso sicuro, verificando le credenziali e rilasciando il token JWT necessario per le chiamate REST successive.
- **Creazione/Aggiornamento Azienda (UC43):** Permette agli attori di business (Produttore, Trasformatore, Distributore) di registrare i propri profili, attivando il workflow di approvazione del Curatore.
- **Visualizzazione Ordini (UC17):** Consente il monitoraggio dello storico acquisti e la gestione transazionale degli ordini attivi.

7.1.3 Scenari di Business

Gli scenari modellati dettagliano i passi operativi per i processi più critici:

- **Gestione del catalogo prodotti (UC44):** Descrive l'inserimento di nuovi articoli e la creazione di pacchetti promozionali da parte del Distributore.
- **Gestione Dinamica del Carrello:** Scenario che coordina l'aggiunta di `MarketplaceItem` e l'applicazione delle strategie di prezzo (`PricingStrategy`) basate sul tipo di articolo selezionato.

7.2 Analisi dei Casi d'Uso (Use Case Diagram)

La modellazione dei casi d'uso definisce le interazioni tra gli attori e il sistema, fornendo una vista funzionale dell'applicazione "Filiera Agricola".

7.2.1 Identificazione degli Attori del Sistema

Oltre agli attori base, il modello Visual Paradigm identifica ruoli specialistici necessari per la gestione della filiera e della qualità dei contenuti:

- **Acquirente:** Utente che interagisce con il Marketplace per acquisti e prenotazioni.
- **Animatore della Filiera:** Responsabile della creazione di eventi promozionali.
- **Gestore della Piattaforma:** Amministratore per la logistica e la configurazione globale.
- **Utente Generico:** Profilo non autenticato con accesso limitato alla sola consultazione.
- **Produttore:** Attore che inserisce i prodotti agricoli primari e ne gestisce la tracciabilità.
- **Trasformatore:** Gestisce le fasi di lavorazione che portano dal prodotto grezzo al prodotto finito.
- **Distributore:** Responsabile della creazione di pacchetti commerciali e della vendita finale.
- **Curatore:** Figura critica per la qualità, incaricata di validare ogni sottomissione di contenuto prima della pubblicazione.
- **Sistemi Esterni:** Integrazione con **Sistema OSM** per la geolocalizzazione e **Sistemi Social** per la condivisione.

7.2.2 Casi d'Uso Principali

Il sistema organizza le funzionalità core per garantire sicurezza e integrità dei dati:

- **Autenticazione (UC12):** Processo di login tramite `AuthController` per il rilascio di token JWT.
- **Creazione/Aggiornamento Azienda (UC43):** Registrazione dei profili aziendali soggetti a validazione.

- **Visualizzazione Ordini (UC17):** Monitoraggio dello stato delle transazioni e dello storico acquisti.
- **Validazione Contenuti:** Funzionalità riservata al Curatore per approvare o rifiutare Prodotti ed Eventi tramite il `CurationService`.

7.2.3 Scenari di Business

Gli scenari descrivono i flussi operativi complessi che coinvolgono più componenti del sistema:

- **Gestione del catalogo prodotti (UC44):** Workflow di inserimento di nuovi articoli nel Marketplace da parte dei distributori.
- **Gestione del Carrello:** Logica di aggiunta articoli e calcolo dinamico dei costi tramite `CarrelloController`.
- **Flusso di Curation (Pattern State):** Gestione del ciclo di vita di un contenuto che transita tra gli stati BOZZA, IN_REVISIONE, APPROVATO o RIFIUTATO.

7.3 Diagramma delle Classi Implementate

Il diagramma delle classi costituisce il nucleo della progettazione logica, definendo la struttura dei dati, il comportamento dei componenti e le relazioni che governano la business logic del sistema.

7.3.1 Architettura dei Controller

L'architettura segue il pattern MVC, delegando ai controller il compito di esporre le API REST e coordinare le chiamate verso lo strato dei servizi.

- **PacchettoController:** Responsabile della gestione dei bundle commerciali. Gestisce le richieste per la creazione di pacchetti promozionali, la consultazione delle offerte attive e l'integrazione con il sistema di *PricingStrategy* per il calcolo dei costi aggregati.
- **CarrelloController:** Gestisce la logica di persistenza temporanea degli acquisti. Espone metodi per l'aggiunta (`aggiungiAlCarrello`), la rimozione e la sincronizzazione degli `ArticoloCarrello` associati alla sessione dell'utente autenticato.
- **AuthController:** Componente critico per la sicurezza del sistema. Si occupa della gestione delle procedure di registrazione di nuovi attori (Produttori, Acquirenti, etc.) e del processo di login, interfacciandosi con `JwtUtils` per il rilascio dei token di autenticazione.

7.3.2 Modellazione delle Entità e Relazioni

Le entità di dominio riflettono la persistenza dei dati sul database relazionale tramite JPA. L'analisi delle molteplicità assicura l'integrità referenziale tra i diversi moduli.

- **Ordine:** Rappresenta la transazione finalizzata. Mantiene una relazione **Many-to-One** con la classe `Utente` (un utente può avere molti ordini) e una relazione **One-to-Many** con `ArticoloOrdine`, che dettaglia le singole righe di acquisto.
- **Azienda:** Entità che modella la struttura produttiva. È associata a un `Utente` con ruolo di `Produttore/Trasformatore` e contiene attributi per la geolocalizzazione e lo stato di validazione (`approvato`).
- **ArticoloCarrello:** Classe di associazione che collega il `Carrello` agli elementi del `MarketplaceItem`. Specifica la quantità desiderata e il riferimento temporale dell'aggiunta.
- **Prodotto:** Entità fondamentale che rappresenta il bene agricolo. Partecipa a relazioni di tracciabilità attraverso i `TraceabilityStep` e può essere inserito in più pacchetti commerciali.
- **Evento:** Modella le iniziative promozionali sul territorio. Possiede una relazione di possesso con un `Animatore` della `Filiera` e una collezione di prenotazioni effettuate dagli acquirenti.
- **Utente:** Classe radice per la gestione dell'identità. È legata a una collezione di `Role` tramite una relazione **Many-to-Many**, permettendo a un singolo account di ricoprire più funzioni operative (es. `Produttore` e `Acquirente` simultaneamente).

L'analisi delle associazioni ha permesso di implementare correttamente i vincoli di *Cascading* (es. la cancellazione di un carrello comporta la rimozione dei relativi `ArticoloCarrello`) e di ottimizzare il caricamento dei dati tramite strategie *Lazy* e *Eager* a seconda della criticità delle query.

7.4 Diagrammi di Interazione (Sequence Diagrams)

I diagrammi di sequenza all'interno del modello UML descrivono il comportamento dinamico del sistema, illustrando lo scambio di messaggi tra gli attori e i componenti software per soddisfare i requisiti funzionali.

7.4.1 Processo di Autenticazione JWT: Flusso di comunicazione per l'accesso sicuro

La modellazione dell'accesso sicuro segue un pattern stateless basato su token JWT. Sebbene il diagramma UML possa semplificare l'interazione in una singola chiamata di `Login()`, l'implementazione effettiva e la logica di design sottostante prevedono un flusso di messaggi articolato tra diversi componenti:

- **Richiesta di Accesso:** Il client invia le credenziali (email e password) all'endpoint `POST /api/auth/login` gestito dall' `AuthController`.
- **Validazione Credenziali:** Il controller delega l'operazione all' `AuthService`, il quale interroga l' `UtenteRepository` per verificare la corrispondenza dell'hash della password (tramite `BCryptPasswordEncoder`).

- **Emissione del Token:** In caso di successo, il servizio utilizza la classe `JwtUtils` per generare un token firmato contenente l'email e i ruoli dell'utente, che viene restituito al client in un oggetto `LoginResponse`.
- **Accesso alle Risorse Protette:** Per ogni richiesta successiva, il client include il token nell'header `Authorization`. Il componente `JwtAuthFilter` intercetta il messaggio, valida il token e imposta il contesto di sicurezza (`SecurityContextHolder`) per autorizzare l'accesso ai controller protetti.

7.4.2 Gestione Dinamica dell'Ordine: Logica di aggiornamento dello stato

Il diagramma di sequenza relativo a "GESTISCI ORDINE" nel file VPP illustra il percorso logico principale (*happy path*) per l'avanzamento dello stato di un acquisto:

- **Innesco dell'Operazione:** Un attore con permessi elevati (come il **Gestore della Piattaforma** o il **Distributore**) interagisce con il sistema per modificare lo stato di un ordine esistente tramite l'endpoint `POST /api/ordini/{id}/stato`.
- **Controllo di Autorizzazione:** Il sistema verifica l'identità dell'attore tramite l'annotazione `@PreAuthorize`, assicurando che solo le figure autorizzate possano procedere con la variazione dello stato logistico.
- **Aggiornamento dello Stato:** L'`OrdineController` invoca il metodo `aggiornaStatoOrdine()` nell'`OrdineService`. Il servizio recupera l'entità `Ordine` dal database e ne modifica il campo di stato (ad esempio, passando da `IN_ELABORAZIONE` a `SPEDITO`).
- **Persistenza e Risposta:** L'entità aggiornata viene salvata tramite l'`OrdineRepository`, e il sistema conferma l'avvenuta operazione all'attore, concludendo l'interazione.

7.5 Design Patterns nel Modello Visuale

L'architettura del sistema integra diversi pattern di progettazione per risolvere problemi ricorrenti legati alla flessibilità del calcolo dei prezzi e alla gestione della logica di business dipendente dallo stato delle entità.

7.5.1 Strategie di Prezzo (Strategy Pattern)

Per gestire le diverse modalità di calcolo del costo degli articoli nel marketplace, è stato implementato lo **Strategy Pattern**. Questa scelta permette di separare gli algoritmi di calcolo dalla classe che li utilizza (`Carrello`), rendendo il sistema facilmente estendibile.

- **PricingStrategyFactory:** Nel modello visuale, questa classe funge da creatore centralizzato. Riceve il tipo di elemento (`Pacchetto` o `MarketplaceItem` singolo) e restituisce l'istanza corretta della strategia di calcolo.
- **PackagePricingStrategy:** Implementazione dedicata al calcolo del prezzo per i bundle commerciali. Gestisce le logiche di sconto applicate ai pacchetti definiti dai Distributori.

- **SingleItemPricingStrategy**: Strategia standard per il calcolo del prezzo dei singoli prodotti agricoli, basata sul prezzo unitario definito dal Produttore o dal Trasformatore.

7.5.2 Ciclo di Vita e Stati: Modellazione delle Entità

La gestione del ciclo di vita delle entità critiche, in particolare per quanto riguarda la tracciabilità e gli ordini, è stata modellata definendo transizioni di stato esplicite. Questo approccio assicura che le operazioni di business siano eseguite solo quando l'entità si trova in una condizione valida.

- **Stati dell'Ordine**: Il modello prevede un'enumerazione dedicata per gestire il workflow degli acquisti. I passaggi principali includono:
 - **IN_ATTESA**: Stato iniziale dell'ordine appena creato, in attesa di conferma del pagamento o validazione.
 - **IN_ELABORAZIONE**: Indica che l'ordine è stato preso in carico dai soggetti della filiera per il confezionamento e la preparazione logistica.
 - **SPEDITO** e **CONSEGNATO**: Stati finali che tracciano il completamento della catena di distribuzione.
- **Ciclo di Vita del Contenuto**: Analogamente, le entità come **Prodotto** ed **Evento** seguono un ciclo di vita che dipende dal ruolo del **Curatore**, passando per lo stato di **APPROVATO** prima di essere visibili pubblicamente nel sistema.

L'utilizzo di questi pattern nel diagramma delle classi permette una traduzione diretta in codice Spring Boot, dove le transizioni di stato sono protette da logiche di validazione nel Service Layer e le strategie di prezzo sono iniettate tramite *Dependency Injection*.

8. Testing, Conclusioni e Sviluppi Futuri

8.1 Strategia di Testing e Validazione

L'infrastruttura di testing è stata progettata e configurata per garantire isolamento, velocità di esecuzione e non-interferenza con i dati di produzione. Sebbene la copertura funzionale dettagliata sia demandata a sviluppi futuri, l'architettura attuale implementa un ambiente di validazione basato su `spring-boot-starter-test`.

8.1.1 Configurazione dell'Ambiente

Per l'esecuzione dei test automatizzati è stato definito un profilo Spring dedicato (`@ActiveProfiles("test")`), che introduce le seguenti caratteristiche architetturali:

- **Database In-Memory:** Sostituzione trasparente del database MySQL di produzione con **H2 Database**. Questo garantisce che ogni sessione di test operi su un dataset effimero, ricreato da zero all'avvio (`create-drop`), eliminando il rischio di *data pollution*.
- **Smoke Testing:** È stato implementato il test fondamentale `contextLoads()`, che valida l'integrità della configurazione del framework. Questo test assicura che il container di Inversion of Control (IoC) riesca a istanziare correttamente tutti i Bean, che la connessione al database H2 sia funzionante e che le dipendenze di sicurezza siano soddisfatte senza conflitti.

Questa configurazione predispone il sistema per l'implementazione futura di test unitari (tramite *JUnit 5* e *Mockito* per il Service Layer) e test di integrazione completi per le API REST, fornendo una base solida per l'adozione di pratiche di Continuous Integration (CI).

8.2 Conclusioni

Il progetto *FilieraAgricola* dimostra come un'architettura software ben progettata possa risolvere problemi complessi di dominio reale. L'uso combinato di pattern architetturali (Layered, REST), pattern di design (State, Strategy) e tecnologie solide (Spring Boot, JPA) ha prodotto un sistema modulare, estendibile e sicuro.

8.3 Sviluppi Futuri

- **Blockchain Integration:** Notarizzare gli hash dei `TraceabilityStep` su una Blockchain pubblica (es. Algorand) per rendere la tracciabilità matematicamente incontestabile.
- **IoT Gateway:** Inserimento automatico dei dati di tracciabilità (temperature, umidità) direttamente dai sensori nei magazzini.

- **Frontend SPA:** Sviluppo di un'interfaccia client completa (React o Angular) per consumare le API prodotte.